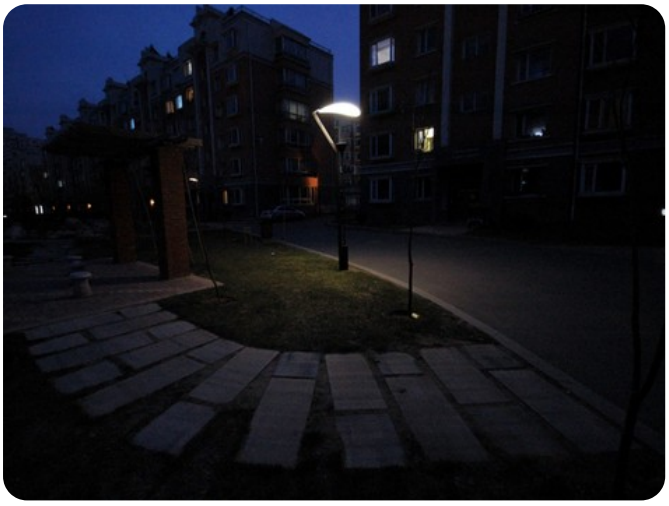
resenh

Robust low-light image restoration and enhancement

Technical solution report. A classical pipeline and a transformer network run side by side, and a quality controller decides, per photo, how much to trust each.



Input, as captured



resenh hybrid: about 6 s in a laptop browser (WebGPU)

Problem ID	P1
Problem title	Robust Low-Light Image Restoration and Enhancement
Team	V-Max (T46), section E
Team members	Sharadh Naidu `1RUA24CSE0421` · Shivani `1RUA24CSE0430` · S. V. Swamy `1RUA24CSE0468` · Sumukha K `1RUA24CSE0483`
GitHub repository	github.com/SharadhNaidu/resen, commit `9cc8c9e`
Live demo	resenh.sharadh.blog (runs in the browser) · pypi.org/project/resenh · Colab notebook

1 Problem understanding and solution strategy

Core technical challenge. A dark photo has very little signal and comparatively a lot of sensor noise. Brightening it amplifies the noise, shifts colour, blows out lamps and windows, and over-brightens scenes that were only partly dark. A fixed curve cannot handle all of these at once: the right amount of gain, denoising and contrast depends on the photo.

Constraints we set ourselves. Results within 10 s on an ordinary laptop or phone with no server; a reproducible, honest evaluation (plain PSNR, no test-set tuning); and an installable package, not only a notebook.

Proposed methodology. We measure the photo first and choose every parameter from those measurements. Two restorers then run in parallel: an adaptive classical pipeline that is predictable and never invents content, and a Retinexformer network that restores colour and texture. A quality controller scores both outputs against the input and blends them with weights that favour the safer one. The hybrid matches the network on clean benchmarks and falls back to the classical result when the network misbehaves.

2 Technical architecture and implementation

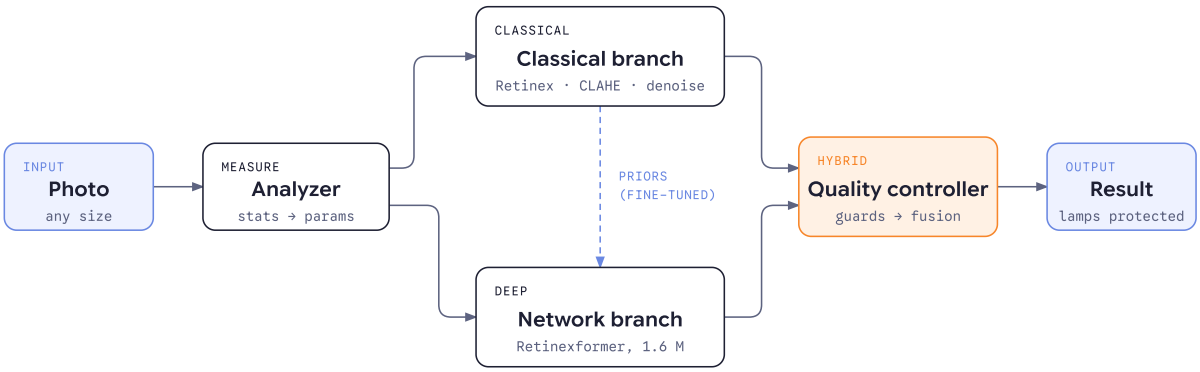


Figure 1. The pipeline (`resenh.enhance`). The analyzer's measurements set the parameters of both branches. The fine-tuned network variant also receives classical priors (illumination map and Retinex output) as extra input channels. The controller (highlighted) produces the final blend. Code: `src/resenh/` — `analysis.py`, `classical/`, `models/`, `controller.py`, `pipeline.py`.

2.1 Classical algorithmic baseline

Algorithms. Illumination is estimated as max-RGB and refined with a guided filter, then used for Retinex/LIME-style brightening (reflectance = image ÷ illumination$^{\gamma}$). This is followed by noise-adaptive guided/bilateral denoising, CLAHE on luma, gray-world white balance and highlight protection.

Adaptive selection logic. The analyzer measures seven statistics, and each parameter is a clipped linear function of one or two of them. Thresholds were set on training and validation images, never on test.

Mean luma, share of dark pixels	→ low-light decision; Retinex γ from 0.85 to 0.45; gain floor
Noise σ (Immerkaer), relative to signal	→ denoise strength 0–1; CLAHE clip from 2.5 down to 1.5
Colour cast (gray-world distance)	→ white-balance strength 0–0.8
Illumination non-uniformity	→ γ relaxed to keep relief in partly lit scenes
Share of near-white highlights	→ highlight protection, strongest for small lamps

2.2 Deep learning pipeline

Model. Retinexformer (ICCV 2023), a 1.6 M-parameter one-stage Retinex transformer. We chose it because it has the best plain-PSNR record of any model with released code. It is also small enough to

run in a browser through ONNX Runtime Web: about 40 MB on first load, then cached.

Loss and training. We fine-tuned the backbone from the official checkpoints. The stem was widened with zero-initialised channels for the classical priors, so training starts exactly at the official model. The loss is Charbonnier + 0.2·(1−SSIM) + 0.02·VGG-19 perceptual, wrapped in the GT-mean loss (ICCV 2025). Training used progressive patches (128→256→384), EMA 0.999, random gamma and noise augmentation, with one model per benchmark: LOL-v1 (485 pairs), LOL-v2-Real (689), LOL-v2-Syn (900). A generalist model was also trained on LOL plus LSRW (5,600 pairs, de-duplicated). Training and validation splits were made by scene so that no scene leaks between them.

2.3 Hybrid integration and adaptive mechanism

Fusion strategy. Classical guidance enters at three points. Before the network, an adaptive noise gate pre-denoises the input in proportion to the measured noise σ. Inside the fine-tuned network, the illumination map and Retinex output are extra input channels. After the network, the controller compares both outputs against the input.

Decision logic. For each candidate the controller computes four guards against the input: newly clipped highlights, chroma shift, edges kept and noise amplification. The guards combine into a penalty, and the weights are $w \propto \text{prior} \cdot \exp(-\text{penalty} / T)$. The prior favours the network on truly dark inputs. Highlight protection then restores lamps the network over-brightened, and inputs that are already bright are blended back towards the original.



Input



Network alone: **orange sky**



Classical branch



Hybrid result

Guard	Network	Classical
Newly clipped highlights	0.4%	0.0%
Colour shift vs input	0.11	0.06
Edges kept	0.47	0.67
Noise amplification	×5.4	×1.4
Fusion weight	**0%**	**100%**

Figure 2. The controller at work on an unseen photo (DICM), with the official LOL-v2-Real backbone as the network candidate. The network amplifies noise ×5 and tints the sky. The controller measures this, gives the classical branch almost all the weight, and caps the gain because the input is not truly dark.

3 Experimental benchmarks and evaluation

Protocol: plain RGB PSNR on the saved 8-bit image, with no ground-truth brightness correction and no per-image knobs. SSIM is MATLAB-style; LPIPS uses AlexNet; NIQE uses pyiqa `niqe_matlab`. Our harness reproduces the official Retinexformer scores exactly (25.15 / 22.79 / 25.67 dB) before comparing anything. Full tables with 95% confidence intervals, noise and edge metrics are in `eval_P1_T46.md`.

3.1 Quantitative benchmarks

LOL-v2-Real test, 100 pairs	PSNR (dB) ↑	SSIM ↑	LPIPS ↓ / NIQE ↓	Latency, ms per MP
Classical baseline	17.44	0.724	0.417 / 4.83	382 GPU 365 CPU
Deep learning only	22.79	0.840	0.171 / 3.59	488 GPU 3774 CPU
Proposed hybrid system	**22.82**	**0.840**	**0.171 / 3.59**	**1906** GPU **4990** CPU

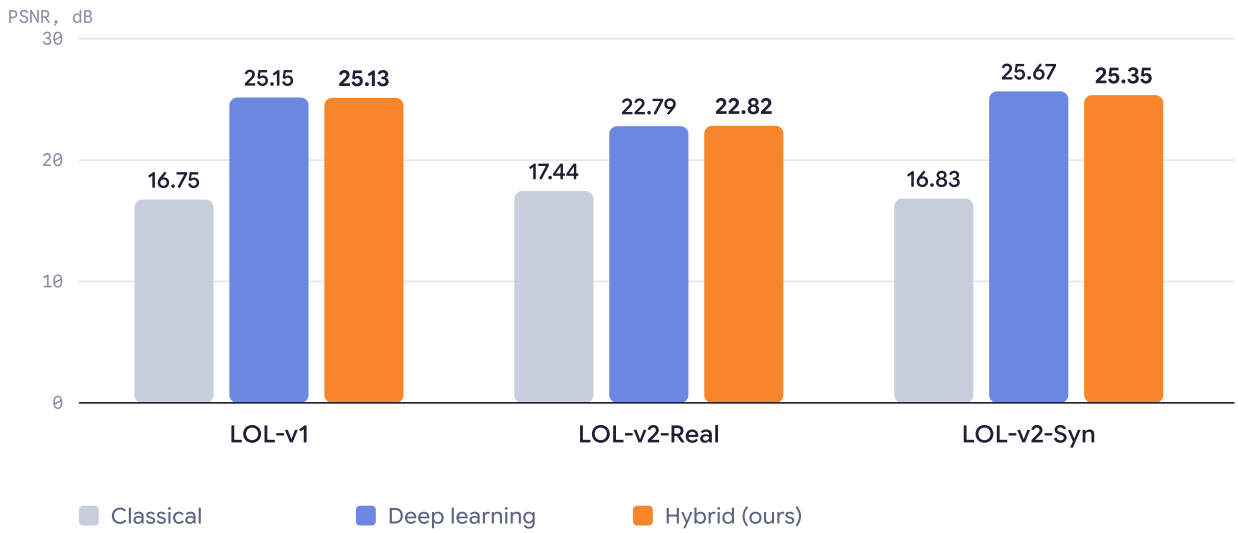


Figure 3. PSNR on the three standard test sets (15, 100 and 100 pairs). The hybrid matches the network within the confidence interval on every set (±0.7 dB on LOL-v2, ±1.5 dB on LOL-v1). The classical branch trails by 6–9 dB, but it never invents content, which is what makes it a useful fallback.

Analysis. Structural fidelity is decided by the network: it adds 5–9 dB PSNR and about 0.1 SSIM over the classical branch, and halves LPIPS. The controller costs essentially nothing on clean benchmark images (+0.03 dB on LOL-v2-Real), but its value appears on real photos (Figure 2). On an RTX 4070 laptop GPU a 1-megapixel photo takes 0.38 s classical, 0.49 s network-only and 1.9 s hybrid. The hybrid includes the classical branch and the priors, which run on the CPU. On the laptop CPU alone the hybrid takes 5.0 s. In a laptop browser the hybrid finishes in about 6 s with WebGPU.

Unseen camera: LSRW

LSRW test, 50 pairs	PSNR	SSIM	LPIPS
Classical	16.15	0.530	0.420
Deep learning only	17.19	0.509	0.331
Hybrid, official backbone	17.32	0.515	0.334
Hybrid, general model (default)	19.26	0.584	0.361

Unseen photos, no ground truth

Set	NIQE, network	NIQE, classical
DICM	3.84	3.21
LIME	4.00	3.33
MEF	3.96	3.52
NPE	4.49	3.71

3.2 Visual outputs and intermediate representations



Input | Classical | Deep learning | **Hybrid** | Ground truth

Figure 4. A LOL-v2-Real test pair. The classical result is brighter but flatter; the network and the hybrid recover the reference's colour.

`Extreme low light (LIME)`



Input | Classical | Deep learning | **Hybrid**

`Backlit scene, bright sky (VV)`



Input | Classical | Deep learning | **Hybrid**

Figure 5. Unseen photos. In the dark scene the lamps stay intact while the shadows open up. In the backlit scene the sky is not blown out and the subjects are lifted.



Illumination | **Gain map** | **Retinex** | **Classical** | **Network** | **Fused result**

Figure 6. Intermediate states for the LIME photo: the illumination map (max-RGB refined by a guided filter), the per-pixel gain, the Retinex output before denoising and CLAHE, both branch outputs and the fused result, as returned by `res.stages` and shown in the web demo's “Every stage” tab.

3.3 Ablation study



Figure 7. PSNR change when one component is switched on, with everything else held fixed. The training ablations are 3,000-iteration fine-tunes from the same initialisation. The GT-mean loss improves SSIM and LPIPS but costs plain PSNR, so the LOL benchmark rows use the official backbone. For real photos the package defaults to the generalist model, which scored 1.5 dB higher on LSRW validation. The noise gate trades 0.55 dB on clean images for 4.3 dB on noisy ones; this is why the package offers a `robust` preset (the default) and a `benchmark` preset.

4 Deployment and quickstart

```
# 1. Install (Python 3.10+; add [dl] for the network, omit it for classical-only)
pip install "resenh[dl]"

# 2. Enhance a folder of photos
resenh enhance ./samples/ -o ./results/            # --mode hybrid | dl | classical

# 3. Or from Python, in Jupyter, Colab or Kaggle
import resenh
res = resenh.enhance("night.jpg")
res.save("night_restored.png"); print(res.summary())

# Reproduce the evaluation
git clone https://github.com/SharadhNaidu/resen && cd resen && pip install -e ".[dl,eval]"
python eval/run_eval.py --bench lolv1 lolv2_real lolv2_syn --methods classical
"net:retinexformer-{bench}" "hybrid-bench:retinexformer-{bench}" --niqe
python eval/make_report.py
```

Deployment formats.

- **Web app:** React and shadcn/ui, running the same Python package in the browser through Pyodide, with the network in ONNX Runtime Web (WebGPU, falling back to WebAssembly). It is hosted as static files on Cloudflare, so there is no server to sleep or queue, and photos never leave the device.
- **Python package and CLI:** `resenh` on PyPI, with weights downloaded on first use (SHA-256 checked) from Hugging Face.
- **Other formats:** a Gradio app (`resenh app`), a Dockerfile, and a Colab/Kaggle quickstart notebook.

Submission checklist

- [x] Public GitHub repository with an updated README
- [x] No raw code pasted into this report
- [x] Every metric is reproducible with `eval/run_eval.py`
- [x] Team details recorded: V-Max, T46, section E